mi faresti il codice python dell'ornstein-ulhenbeck multidimensionale delle cellule di collagene che si avvicinano al tumore celebrale glioblastoma?

Ti propongo un esempio **generico e puramente matematico** di processo di **Ornstein–Uhlenbeck multidimensionale** applicato a particelle che vengono attratte verso un punto fisso (che puoi interpretare come il centro del tumore).

⚠️ Nota: questo è solo un modello stocastico astratto per simulazione numerica. Non è un modello biologico realistico del comportamento cellulare.

📌 Modello matematico

Processo OU in Rd\mathbb{R}^dRd:

dXt=θ(μ−Xt) dt+σ dWtd\mathbf{X}_t = \theta (\mu - \mathbf{X}_t)\, dt + \sigma \, d\mathbf{W}_tdXt=θ(μ−Xt)dt+σdWt

- Xt\mathbf{X}_tXt = posizione della cellula
- μ\muμ = punto di attrazione (centro tumore)
- θ\thetaθ = intensità attrattiva
- σ\sigmaσ = rumore
- dWtd\mathbf{W}_tdWt = moto browniano multidimensionale

🐍 Codice Python (Euler–Maruyama)

```
import numpy as np

import matplotlib.pyplot as plt


def ornstein_uhlenbeck_md(

    n_cells=50,

    dim=2,

    theta=1.0,

    sigma=0.5,

    mu=None,

    T=10.0,

    dt=0.01
```

```
):
    """
    Simulazione di un processo Ornstein-Uhlenbeck multidimensionale
    per n_cells particelle.
    """

    n_steps = int(T / dt)

    if mu is None:
        mu = np.zeros(dim)

    # Posizioni iniziali casuali
    X = np.random.randn(n_cells, dim)

    # Salvataggio traiettorie
    trajectories = np.zeros((n_steps, n_cells, dim))
    trajectories[0] = X

    for t in range(1, n_steps):
        dW = np.sqrt(dt) * np.random.randn(n_cells, dim)
        X = X + theta * (mu - X) * dt + sigma * dW
        trajectories[t] = X

    return trajectories


# Parametri simulazione
dim = 2
mu = np.array([0.0, 0.0])  # centro del tumore
```

```
traj = ornstein_uhlenbeck_md(
    n_cells=30,
    dim=dim,
    theta=0.8,
    sigma=0.4,
    mu=mu,
    T=20,
    dt=0.01
)

# Visualizzazione
plt.figure(figsize=(6,6))
for i in range(traj.shape[1]):
    plt.plot(traj[:, i, 0], traj[:, i, 1], alpha=0.6)

plt.scatter(mu[0], mu[1], c='red', s=100, label='Centro tumore')
plt.title("Ornstein-Uhlenbeck multidimensionale")
plt.legend()
plt.xlabel("x")
plt.ylabel("y")
plt.axis("equal")
plt.show()
```

🔬 **Cosa fa il codice**

- Simula più cellule come particelle stocastiche
- Ogni cellula è attratta verso mu (centro del tumore)
- Il termine sigma introduce fluttuazioni casuali
- theta controlla la forza di richiamo

Se vuoi posso:

- Estenderlo a 3D
- Aggiungere interazione tra cellule
- Inserire un potenziale radiale più realistico
- Oppure scriverlo in forma vettorializzata più efficiente per molte cellule

potresti farlo animato?

Certo 🙂
Qui sotto trovi una versione **animata** del processo di Ornstein–Uhlenbeck multidimensionale (2D) usando matplotlib.animation.

⚠️ È sempre un modello matematico astratto (attrazione verso un punto fisso + rumore).

🐍 Codice Python animato (2D)

```
import numpy as np

import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation


def ornstein_uhlenbeck_md(

    n_cells=50,

    dim=2,

    theta=1.0,

    sigma=0.5,

    mu=None,

    T=10.0,

    dt=0.02

):

    n_steps = int(T / dt)


    if mu is None:
```

```
        mu = np.zeros(dim)

    X = np.random.randn(n_cells, dim)
    trajectories = np.zeros((n_steps, n_cells, dim))
    trajectories[0] = X

    for t in range(1, n_steps):
        dW = np.sqrt(dt) * np.random.randn(n_cells, dim)
        X = X + theta * (mu - X) * dt + sigma * dW
        trajectories[t] = X

    return trajectories


# Parametri
n_cells = 40
mu = np.array([0.0, 0.0])
traj = ornstein_uhlenbeck_md(
    n_cells=n_cells,
    dim=2,
    theta=0.8,
    sigma=0.6,
    mu=mu,
    T=20,
    dt=0.02
)

# Setup figura
fig, ax = plt.subplots(figsize=(6,6))
```

```
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_title("OU multidimensionale animato")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_aspect("equal")

# Punto del tumore
ax.scatter(mu[0], mu[1], c='red', s=120, label='Centro tumore')

# Scatter iniziale cellule
scat = ax.scatter(traj[0,:,0], traj[0,:,1], c='blue', s=30)

def update(frame):
    scat.set_offsets(traj[frame])
    return scat,

anim = FuncAnimation(
    fig,
    update,
    frames=len(traj),
    interval=30,
    blit=True
)

plt.legend()
plt.show()
```

🎬 Se vuoi salvare il video

Aggiungi alla fine:

anim.save("ou_simulation.mp4", fps=30)

(Serve ffmpeg installato.)

🔬 Cosa vedrai

- Le cellule si muovono casualmente
- Ma tendono statisticamente a convergere verso il centro
- Se aumenti theta → attrazione più forte
- Se aumenti sigma → più rumore e traiettorie più caotiche

Se vuoi posso anche:

- Far vedere le **traiettorie complete in fading**
- Aggiungere **interazione tra cellule**
- Inserire un **potenziale radiale non lineare**
- Oppure fare una versione 3D animata 🚀

potresti visualizzare anche le traiettorie?

Perfetto 🙂
Qui sotto trovi una versione animata in cui:

- I punti blu = cellule
- Il punto rosso = centro di attrazione
- Le linee sottili = traiettorie complete
- Le linee più evidenti = scia “recente” (effetto fading)

🐍 OU multidimensionale animato con traiettorie

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation

```
def ornstein_uhlenbeck_md(
    n_cells=30,
    dim=2,
    theta=1.0,
    sigma=0.5,
    mu=None,
    T=15.0,
    dt=0.02
):
    n_steps = int(T / dt)

    if mu is None:
        mu = np.zeros(dim)

    X = np.random.randn(n_cells, dim)
    trajectories = np.zeros((n_steps, n_cells, dim))
    trajectories[0] = X

    for t in range(1, n_steps):
        dW = np.sqrt(dt) * np.random.randn(n_cells, dim)
        X = X + theta * (mu - X) * dt + sigma * dW
        trajectories[t] = X

    return trajectories


# Parametri
n_cells = 20
```

```
mu = np.array([0.0, 0.0])
traj = ornstein_uhlenbeck_md(
    n_cells=n_cells,
    dim=2,
    theta=0.9,
    sigma=0.6,
    mu=mu,
    T=20,
    dt=0.02
)

n_steps = traj.shape[0]

# Setup figura
fig, ax = plt.subplots(figsize=(6,6))
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_aspect("equal")
ax.set_title("OU multidimensionale con traiettorie")
ax.set_xlabel("x")
ax.set_ylabel("y")

# Centro attrattivo
ax.scatter(mu[0], mu[1], c='red', s=120, label='Centro tumore')

# Scatter cellule
scat = ax.scatter(traj[0,:,0], traj[0,:,1], c='blue', s=40)

# Linee traiettorie complete (sfondo leggero)
```

```
full_lines = []
for i in range(n_cells):
    line, = ax.plot([], [], lw=0.5, alpha=0.3)
    full_lines.append(line)

# Linee scia recente
trail_length = 40
trail_lines = []
for i in range(n_cells):
    line, = ax.plot([], [], lw=1.5)
    trail_lines.append(line)

def update(frame):
    # aggiorna punti
    scat.set_offsets(traj[frame])

    for i in range(n_cells):
        # traiettoria completa
        full_lines[i].set_data(
            traj[:frame, i, 0],
            traj[:frame, i, 1]
        )

        # scia recente
        start = max(0, frame - trail_length)
        trail_lines[i].set_data(
            traj[start:frame, i, 0],
            traj[start:frame, i, 1]
        )
```

```
    return [scat] + full_lines + trail_lines


anim = FuncAnimation(
    fig,
    update,
    frames=n_steps,
    interval=30,
    blit=True
)

plt.legend()
plt.show()
```

🔬 **Cosa ottieni**

- Movimento stocastico tipo Browniano
- Drift verso il centro
- Visualizzazione completa delle traiettorie
- Effetto “memoria corta” tramite scia mobile

Se vuoi posso anche:

- Colorare le traiettorie in base alla distanza dal centro
- Far decadere l’opacità nel tempo
- Inserire un campo di potenziale visibile come mappa di calore
- Rendere il sistema più biologicamente ispirato (gradiente chimico non lineare)